Advanced topic 1: Version Control Workflows (Git)

Link to our GitHub: janufk/BlackJack: BlackJack

We chose this advanced topic because it is a convenient way to share code with each other. GitHub also provides us a place where all current and past changes are seen. This gives us a way to see our improved code, but it also works as a backup tool. Lastly, it is a good skill to learn for the future, since it is common to collaborate with others through GitHub.

We learned different things about Git before using it. Our research was done by looking up several YouTube tutorials. One of the most important things is how to move the code from one laptop to another. Pushing the code to Git is done with the following steps: first, commit your project. Then make a personal token in GitHub (which was also new to us). Lastly, push to upstream through remote. Pulling the code to get it in Java is done with pull from upstream. Before doing all of this, Git has to connect to Apache NetBeans, which is also a new skill we acquired.

We applied this topic by using Git to share our code. This is done by both using the push and pull function. We also reviewed previous committed code to compare it with our current code and to further improve it.

Overall, it was very helpful for our collaboration. We could work on the code, even when we were not together. The one roadblock for us was not knowing when a force push is needed. We came across this problem when one of us wanted to push without pulling the new code first. After some researching, we discovered how to do a force push. This was not clear from Apache NetBeans nor Git, which stirred up confusion.

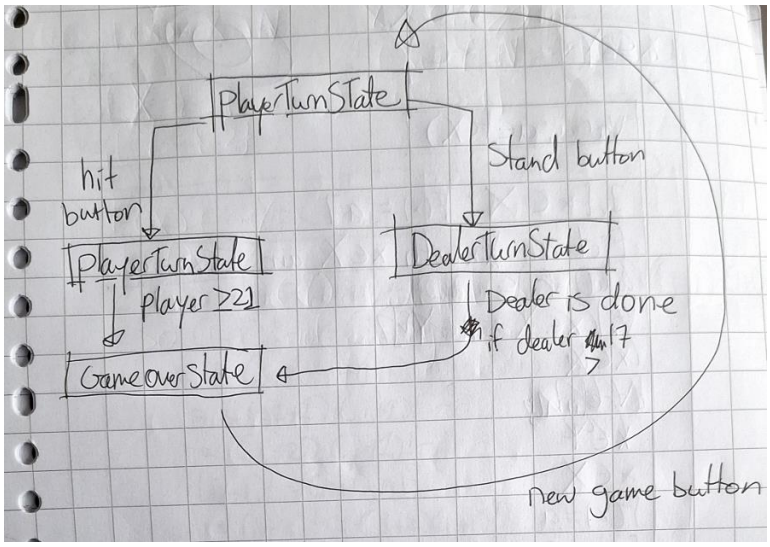
Advanced topic 2: Advanced Object-Oriented Design Patterns (State pattern)

We looked at different types of design patterns to use for our blackjack code. After looking at the game flow, we concluded that state pattern would be the best choice. This is because a real Blackjack game has different states. A player plays and either hits or stands. After this, it is the dealers turn and the game ends. Because actions can be made depending on the current state of the game, the state pattern is the best fit in our opinion.

During our research, we learned a lot of new things about state pattern. It is a design pattern that changes the behaviour of an object, depending on its current state. Each state has its own rules and responsibilities. In our case, we wanted to have a state of a player, dealer and when the game is done.

We implemented the state pattern by starting with a GameState. We defined a GameState interface with hit() and stand(), and three concrete states. The first one is PlayerTurnState, where it handles the player's actions. Second one is DealerTurnState, where it handles the dealer's actions. The last one is GameOverState, which represents the end of a game round.

Since we have never worked with a state pattern, it was difficult how to determine good states. After making a mind map (see photo below) and receiving feedback on it, we could get started. At the end, it is a good and helpful way to separate functions. It helped us having a clear overview of the code and game logic.



Bibliography

corbin. (2025, March 13). *How To Use GitHub For Beginners*. Retrieved from YouTube: https://www.youtube.com/watch?v=a9u2yZvsqHA

Geekific. (2021, September 11). *The State Pattern Explained and Implemented in Java | Behavioral Design Patterns | Geekific*. Retrieved from YouTube: https://www.youtube.com/watch?v=abX4xzaAsoc

Geeksforgeeks. (2025, July 23). *How to Force Git Push?* Retrieved from Geeksforgeeks: https://www.geeksforgeeks.org/git/how-to-force-git-push/

Geeksforgeeks. (2026, April 28). *Introduction to GitHub*. Retrieved from Geeksforgeeks: https://www.geeksforgeeks.org/git/introduction-to-github/

Geeksforgeeks. (2026, May 13). *State Design Pattern*. Retrieved from Geeksforgeeks: https://www.geeksforgeeks.org/system-design/state-design-pattern/

guru, R. (n.d.). *State in Java*. Retrieved from Refactoring guru: https://refactoring.guru/design-patterns/state/java/example